[10 주차] RAG 원리 데모

강사용 코드 해설본 — NotebookLM 블랙박스 안에서 실제로 벌어지는 일

1. 이 노트북의 성격 — 학생용이 아니라 시연용

먼저 짚어둘 점: 이 노트북은 3 주차 노트북들과 달리 학생이 직접 코딩하는 실습이 아닙니다. 노트북 맨 앞에 "학생이 코딩할 필요는 없고, 강사가 실행하며 원리를 설명하는 시연용"이라고 명시돼 있습니다. 즉 김사부님이 강의실에서 이 코드를 대신 실행해 보여주고, 화면에 뜨는 결과만으로 RAG 의 3 단계(청크 분할 → 임베딩·검색 → 생성)를 설명하시면 되는 구조입니다.

그래서 이 해설본의 목적도 3 주차 해설본과 다릅니다 — "코드를 줄줄이 읽게" 하려는 게 아니라, "이 셀을 실행하면 화면에 뭐가 뜨고, 그게 왜 중요한지"를 강의 멘트로 바로 쓰실 수 있게 만드는 것입니다.

2. 큰 그림 먼저 — RAG 가 뭔가

RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 "AI 가 아무 말이나 지어내지 않고, 미리 준비된 자료(소스) 안에서만 근거를 찾아서 답하게 만드는 방법"입니다. 우리가 매주 쓰는 NotebookLM 이 바로 이 방식으로 동작합니다 — 사용자가 올린 PDF 나 문서 안에서만 답을 찾고, "출처"를 표시해주는 것이 그 증거입니다.

🎯 **핵심 포인트** 이 노트북 전체는 결국 한 문장으로 요약됩니다: "질문과 의미가 비슷한 문장을 자료 더미에서 찾아낸 뒤, 그 문장들만 근거로 삼아 답을 만든다." 이 노트북의 6 단계는 그 한 문장을 코드로 하나씩 풀어서 보여주는 것뿐입니다.

3. 1 단계 — 라이브러리 설치

```
!pip install -q langchain langchain-community langchain-huggingface
                  langchain-text-splitters faiss-cpu sentence-transformers
```

LangChain 은 RAG 같은 "LLM 을 활용한 파이프라인"을 만들 때 업계 표준처럼 쓰이는 도구 모음입니다. FAISS 는 페이스북(메타)이 만든 "벡터를 빠르게 검색해주는 저장소" 도구, sentence-transformers 는 문장을 벡터로 바꿔주는 모델 도구입니다. 이 셀은 그 도구들을 코랩에 설치하는 것뿐이라 설명할 내용은 많지 않습니다.

4. 2 단계 — 지식베이스 만들기

여기서 만드는 것이 바로 NotebookLM 에 "소스로 업로드"하는 문서와 같은 역할입니다. 추석 연휴 기상예보 문장들을 한 줄씩 잘라서 "문서" 목록으로 만듭니다.

```
from langchain_core.documents import Document

facts_raw = """
2026 년 9 월 16 일 18 시, 기상청은 추석 연휴 중기예보를 발표했다.
서울·인천·경기 지역은 9 월 24 일 최저기온 16 도, 최고기온 29 도...
"""
```

```
docs = [Document(page_content=line.strip())
        for line in facts_raw.strip().split("\n") if line.strip()]
```

코드	무슨 뜻인가
`Document(page_content=...)`	LangChain 이 "이건 하나의 완결된 문서 조각이다"라고 인식하게 포장하는 상자입니다. 우리가 쓰는 문장 하나하나가 이 상자 하나씩에 담깁니다.
`facts_raw.strip().split("\\n")`	긴 텍스트 덩어리를 줄바꿈(\\n) 기준으로 쪼개서, 문장 하나하나를 리스트로 만듭니다. "신문 기사를 한 문장씩 가위로 오려낸다"고 보시면 됩니다.
`[... for line in ... if line.strip()]`	빈 줄은 건너뛰면서, 남은 문장들을 전부 Document 상자에 담아 docs 라는 리스트로 모읍니다.

💡 **강의 팁** 이 자료가 "검증된 정답 데이터"라는 점이 핵심입니다. RAG 는 AI 가 아무 데서나 정보를 가져오는 게 아니라, 우리가 미리 골라둔 신뢰할 수 있는 자료 안에서만 찾는다는 걸 이 셀에서 보여주는 것입니다.

5. 3 단계 — 청크 분할 (Chunking)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=100, chunk_overlap=10)
chunks = splitter.split_documents(docs)
```

AI 는 문서를 통째로 읽지 않고 작은 조각(청크)으로 잘라서 다룹니다. 지금 예시는 문장이 이미 짧아서 큰 변화가 없지만, 실제로 긴 뉴스 기사 한 편을 넣으면 여러 개의 청크로 쪼개지는 과정을 이 코드가 담당합니다.

코드	무슨 뜻인가
`chunk_size=100`	청크 하나의 최대 글자수를 100 자로 제한합니다. 너무 길면 AI 가 그 안에서 원하는 부분을 정확히 못 찾아서, 적당한 크기로 잘라두는 것입니다.
`chunk_overlap=10`	청크와 청크 사이를 10 자씩 겹치게 자릅니다. 문장이 애매한 경계에서 뚝 끊기면 의미가 손실될 수 있어서, 앞뒤를 살짝 겹쳐 안전장치를 두는 것입니다.

💡 **강의 팁** "청크"를 "신문 스크랩"에 비유하면 이해가 빠릅니다 — 기사 전체를 벽에 붙이는 대신, 필요한 문단만 오려서 인덱스카드처럼 만들어두는 것과 같습니다. chunk_overlap 은 "오려낼 때 옆 문장 한 줄을 살짝 걸쳐서 자르는 것"에 해당합니다.

6. 4 단계 — 임베딩 + 벡터스토어 구축

이 단계가 RAG 에서 가장 낯설면서도 가장 핵심적인 부분입니다. 문장을 "의미를 담은 숫자 배열"로 바꾸는 과정입니다.

```
from langchain_huggingface import HuggingFaceEmbeddings
```

```
from langchain_community.vectorstores import FAISS

embeddings = HuggingFaceEmbeddings(model_name="...MiniLM-L12-v2")
vectorstore = FAISS.from_documents(chunks, embeddings)

sample_vector = embeddings.embed_query(chunks[0].page_content)
print(f"{len(sample_vector)}개의 숫자로 이루어진 벡터로 변환됐습니다")
```

코드	무슨 뜻인가
`HuggingFaceEmbeddings(model_name=...)`	허깅페이스(HuggingFace)는 AI 모델들을 무료로 받아 쓸 수 있는 저장소 사이트입니다. 여기서는 "문장을 벡터로 바꿔주는 전용 모델"을 하나 불러오는 것입니다. multilingual 이라는 이름에서 알 수 있듯 한국어도 지원합니다.
`FAISS.from_documents(chunks, embeddings)`	앞서 만든 청크들을 전부 벡터로 바꾼 뒤, 빠르게 검색할 수 있는 저장소(FAISS)에 넣습니다. 이 한 줄이 사실상 "검색엔진을 하나 만드는 것"과 같습니다.
`embeddings.embed_query(...)`	문장 하나를 실제로 벡터로 바꿔서 눈으로 확인해보는 코드입니다. 결과로 "384 개의 숫자로 이루어진 벡터" 같은 출력이 나옵니다.

🎯 **핵심 포인트** "벡터"란 결국 "그 문장의 의미를 수백 개의 숫자 좌표로 찍어놓은 것"입니다. 의미가 비슷한 문장은 이 좌표들이 서로 가까운 곳에 찍히고, 의미가 다른 문장은 멀리 찍힙니다. 그래서 "의미가 비슷한 문장을 찾는다"는 것이 실제로는 "좌표상 가까운 점을 찾는다"는 수학 문제로 바뀝니다 — 이게 바로 AI 가 "이해"하는 것처럼 보이는 트릭의 정체입니다.

💡 **강의 팁** 비유: 도서관 사서가 책을 "내용"이 아니라 "청구기호(숫자 좌표)"로 배열해두면, 비슷한 주제의 책들이 자연스럽게 옆자리에 꽂히는 것과 같습니다. 임베딩은 모든 문장에 이런 청구기호를 매기는 작업입니다.

7. 5 단계 — 검색 시연 (수업 시간의 핵심 장면)

여기가 이 노트북에서 실제로 학생들에게 "아하" 하는 순간을 만들어주는 부분입니다. 일부러 두 가지 질문을 던집니다.

- 질문 1: "9 월 25 일 광주전남 최고기온은 몇 도야?" → 소스에 실제로 있는 사실
- 질문 2: "기상청 관계자가 코멘트했어?" → 소스에 없는, 예전 실습에서 AI 가 지어냈던(할루시네이션) 바로 그 내용

```
results1 = vectorstore.similarity_search_with_score("9 월 25 일 ...", k=2)
for doc, score in results1:
    print(f"[유사도 거리 {score:.3f}] {doc.page_content}")
```

코드	무슨 뜻인가
`similarity_search_with_score(질문, k=2)`	입력한 질문 문장도 벡터로 바꾼 뒤, 저장소 안에서 그 벡터와 가장 가까운 청크 2 개(k=2)를 찾아옵니다. "유사도 거리(score)"도 같이 돌려줍니다.

코드	무슨 뜻인가
`score (유사도 거리)`	숫자가 작을수록 질문과 더 관련 있는 문장이라는 뜻입니다. (거리이기 때문에 가까울수록 작은 숫자입니다)

🎯 **핵심 포인트** 실행해보면 질문 1 의 유사도 거리는 작고(관련 있음), 질문 2 의 유사도 거리는 뚜렷하게 큽니다(관련 없음). 즉 시스템은 "이건 근거가 없다"는 신호를 이미 갖고 있었습니다. 그런데 예전 실습에서 만든 도메인별 챗봇은 이 신호를 무시하고 관계자 코멘트를 지어냈던 것입니다 — 이게 바로 할루시네이션이 발생하는 지점을 코드 레벨에서 보여주는 장면입니다.

💡 **강의 팁** 수업 멘트 제안: "AI 가 거짓말을 한 게 아니라, '모르겠다'고 말했어야 할 순간에 그 신호를 무시하도록 설계됐기 때문에 지어낸 겁니다. 지금 화면의 숫자가 바로 그 증거입니다." — 이 한 장면만으로 지난 실습에서 본 할루시네이션을 기술적으로 설명할 수 있습니다.

8. 6 단계(선택) — 검색 결과로 실제 답변 생성하기

여기까지는 "검색(Retrieval)"만 한 것이고, 실제 NotebookLM 은 검색된 청크를 근거로 자연스러운 문장 답변을 "생성(Generation)"하는 단계가 하나 더 있습니다. 이 단계는 Gemini API 키가 있어야 실행되며, 키가 없으면 건너뛰어도 5 단계까지의 설명으로 충분합니다.

```
def rag_answer(question, k=4):
    found = vectorstore.similarity_search(question, k=k)
    context = "\n".join(d.page_content for d in found)
    prompt = f"다음 근거만 사용해서 질문에 답하세요.
             근거에 답이 없으면 반드시 '모른다'고 답하세요.
             근거:\n{context}\n\n 질문: {question}"
    return llm.invoke(prompt).content
```

코드	무슨 뜻인가
`found = vectorstore.similarity_search(...)`	5 단계와 똑같이 질문과 비슷한 청크들을 찾아옵니다.
`context = "\\n".join(...)`	찾아온 청크 여러 개를 하나의 긴 텍스트로 이어 붙입니다. 이게 LLM 에게 줄 "근거 자료"가 됩니다.
`prompt = f"다음 근거만 사용해서..."`	LLM 에게 보낼 지시문(프롬프트)을 만듭니다. "내가 주는 근거 안에서만 답하고, 없으면 모른다고 해라"는 규칙을 문장으로 명시하는 것이 핵심입니다.
`llm.invoke(prompt).content`	완성된 프롬프트를 실제 LLM(Gemini)에게 보내고, 돌아온 답변 텍스트만 꺼냅니다.

🎯 **핵심 포인트** 이 노트북 전체에서 가장 중요한 한 줄은 프롬프트 안의 "근거에 답이 없으면 반드시 '모른다'고 답하세요"입니다. RAG 가 할루시네이션을 막아주는 건 저절로 되는 마법이 아니라, 이렇게 사람이 명시적으로 지시문에 적어야 작동하는 "설계"라는 점을 학생들에게 꼭 짚어주시면 좋습니다.

9. 정리 — 학생들에게 남길 두 문장

- NotebookLM 이 '출처를 표시'할 수 있는 이유는, 답변을 만들기 전에 이미 '어느 청크에서 가져왔는지'를 알고 있기 때문입니다.
- 관련 근거가 없을 때 '모른다'고 답하게 만드는 것은 저절로 되는 게 아니라, 프롬프트 안에 명시적으로 설계해야 하는 부분입니다. 이 설계가 부실하면 할루시네이션이 발생합니다.

10. 용어 사전

용어	쉬운 설명 (학생에게 쓸 수 있는 말)
RAG	검색 증강 생성. "미리 준비한 자료 안에서 근거를 찾은 뒤, 그 근거로만 답을 만드는 방식".
임베딩 (Embedding)	문장의 의미를 수백 개의 숫자 좌표(벡터)로 바꾸는 작업. 의미가 비슷하면 좌표도 가깝게 찍힙니다.
벡터 (Vector)	임베딩의 결과물. 숫자들의 배열로, 문장의 "의미 지문"이라고 생각하시면 됩니다.
청크 (Chunk)	긴 문서를 다루기 좋은 크기로 잘라낸 조각. 신문 기사를 오려낸 스크랩 한 장에 비유할 수 있습니다.
벡터스토어 (Vector Store)	수많은 벡터를 저장해두고, "이 벡터와 가장 가까운 것들"을 빠르게 찾아주는 전용 저장소. (예: FAISS)
유사도 검색 (Similarity Search)	질문 벡터와 가장 가까운(=의미가 비슷한) 청크들을 찾아오는 작업.
유사도 거리 (Score)	두 벡터가 얼마나 가까운지 나타내는 숫자. 작을수록 더 관련 있다는 뜻.
LangChain	RAG 처럼 여러 단계(청크 분할, 임베딩, 검색, 생성)를 이어붙인 파이프라인을 만들 때 표준처럼 쓰이는 도구 모음.
LLM	거대언어모델. ChatGPT, Gemini 처럼 문장을 이해하고 생성하는 AI 모델.
프롬프트 (Prompt)	LLM 에게 내리는 지시문. "근거 안에서만 답해라" 같은 규칙도 이 지시문 안에 글로 적어야 작동합니다.
할루시네이션 (Hallucination)	AI 가 근거 없이 그럴듯한 거짓 정보를 지어내는 현상.

11. 강의 진행 팁 — 실제로 어떻게 시연할까

- 사전 준비: 코랩에서 이 노트북을 미리 한 번 끝까지 실행해두고, 임베딩 모델 다운로드(1~2 분)는 수업 시작 전에 끝내두시면 강의 흐름이 끊기지 않습니다.
- 가장 임팩트 있는 장면은 7 장(5 단계)의 유사도 거리 비교입니다. 여기에 수업 시간을 가장 많이 배분하시는 걸 추천드립니다.
- Gemini API 키가 없으면 6 단계는 통째로 건너뛰어도 무방합니다 — 노트북 자체에도 "5 단계까지의 원리 설명으로 충분하다"고 명시돼 있습니다.

- 직전 실습(도메인 챗봇)에서 AI 가 관계자 코멘트를 지어냈던 사례를 다시 화면에 띄워놓고, 이 노트북의 질문 2 와 나란히 비교해서 보여주면 "그때 그 문제가 바로 이거였다"는 연결이 확실해집니다.

※ 이 문서는 10 주차_RAG 원리_데모_코랩.ipynb 의 실제 코드를 근거로 작성되었습니다. 노트북은 학생용 실습이 아니라 강사 시연용이므로, 코드를 직접 타이핑하실 필요 없이 그대로 실행하시면서 이 해설본의 멘트를 참고하시면 됩니다.